Novelty 1: Sales, Cost and Profit Forecasting

Panel presentation guide - what to say, in simple words

SmartCost: Business Performance Prediction System for Micro-SMEs

K.A.C.H. Kodithuwakku (IT22544872) - Group R26-IT-026

How to use this guide

- **Blue boxes** = sentences you can say to the panel almost word for word.
- **Green boxes** = the simple meaning, so you understand it yourself before you explain it.
- **Yellow boxes** = be careful: things not to claim, or limits to admit honestly.
- Section 12 has the likely panel questions with short answers.
- Section 13 is a 1-page cheat sheet with all the numbers.

Suggested talk plan (about 7 minutes)

Time	Part	Section
0:00 - 0:45	Problem and research gap	1, 2
0:45 - 1:30	My novelty in one sentence + the 6 parts	3
1:30 - 3:00	How it works (diagram) and each part simply	4, 5
3:00 - 4:30	How I tested it and the results	6, 7
4:30 - 6:00	Live demo in the app	9
6:00 - 7:00	Limitations and future work	10, 11

1. The problem

Small shops (restaurants, juice bars, bakeries) use SmartCost to record sales and expenses. But the owner still cannot answer simple questions about the future:

- How much will I sell next month, and in the next 3, 6 or 12 months?
- How much will I spend?
- Will I make a profit or a loss?
- How much do I need to sell just to cover my costs?

Big forecasting tools exist, but they assume **long, clean data** and an **expert** to run them. A micro-SME has neither: often only a few months of history, gaps where staff forgot to enter sales, and nobody who knows statistics.

SAY THIS TO THE PANEL

Micro-SME owners record their sales every day, but they still cannot see next month's sales, costs or profit. Existing forecasting tools need long, clean data and an expert. Small shops have short, messy records and no analyst. My novelty closes that gap.

2. The research gap

What existing tools usually do	What a micro-SME actually needs
Assume years of complete data	Work from 3 months, improve as data grows
Use one fixed model for everyone	Pick what works for this shop's data
Ignore missing or half-entered months	Detect bad months and repair them
Show a single number	Show how sure the forecast is (a range)
Forecast sales only	Sales, costs, profit, loss risk and break-even
Technical output (charts, error metrics)	Plain sentences an owner understands

SIMPLE MEANING

Other tools say 'here is a number'. Mine says 'here is a number, this is how sure I am, this is your chance of a loss, and by the way two of your months look incomplete'.

3. My novelty in one sentence

SAY THIS TO THE PANEL

An adaptive forecasting engine built into a POS system that adjusts to how much data a small shop has and how good that data is, and turns the forecast into decisions an owner understands: a profit range, the chance of a loss, the break-even point and how this month is tracking.

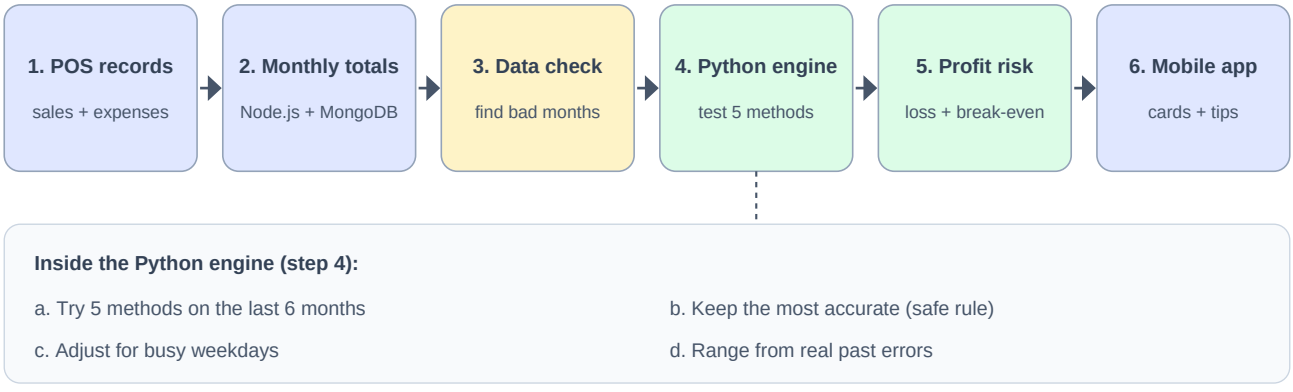
The 6 parts of the novelty

#	Part	What it does (simple)
1	Data-quality check	Finds months that were not fully recorded and fixes them before forecasting.
2	Adaptive model choice	Tests 5 methods on the shop's own recent months and keeps the best one, safely.
3	Weekday calendar	Learns the shop's busy weekdays; a month with five Saturdays is forecast higher.
4	Honest ranges	The 'between X and Y' range comes from the model's real past mistakes.
5	Profit risk + break-even	Gives a profit range, the chance of a loss, and the sales needed to cover costs.
6	Owner-friendly output	Plain-English tips and 'this month is ahead / on track / behind'.

SAY THIS TO THE PANEL

Each part solves a real problem of micro-SME data: short history, missing records, and owners who need decisions, not statistics.

4. How it works



SAY THIS TO THE PANEL

Every sale and expense from the POS is grouped into monthly totals. Before forecasting, the system checks the data and repairs incomplete months. Then the Python engine tests five forecasting methods on the shop's own recent months, keeps the most accurate one, adjusts for busy weekdays and measures how wrong it usually is. Finally it calculates profit risk and break-even, and the app shows everything in simple cards.

Technology: Expo React Native app, Node.js / Express API, MongoDB, and a Python FastAPI service using statsmodels.

5. Each part explained simply

Part 1 - Data-quality check

The system compares every month with the shop's **normal month** (the median). A month is flagged when it has fewer than 40% of the normal number of orders, fewer than half the normal selling days, or no expenses at all.

- Bad month at the **end** (for example the last month was half entered): it is left out, and the model estimates it instead.
- Bad month in the **middle**: it is filled in from the months before and after.
- The owner sees a message: 'Aug 2026 looks incompletely recorded, so it was left out.'

SIMPLE MEANING

If staff forget to enter sales for two weeks, a normal model thinks business crashed and predicts almost zero. My system notices the month is incomplete and does not learn from it.

SAY THIS TO THE PANEL

In my evaluation, one half-recorded month made the old model predict zero sales, with a back-test error of over 8,000%. The new data-quality check detects such months automatically and leaves them out.

Part 2 - Adaptive model choice

Five forecasting methods are tried:

Method	Simple meaning	Needs
Moving average	Average of the last 3 months	3 months
Seasonal naive	Same as the same month last year	12 months
Holt (damped trend)	Follows the trend up or down, slowly levelling off	4 months
Holt-Winters additive	Trend + yearly pattern (e.g. December is busy)	24 months
Holt-Winters multiplicative	Same, but the yearly swing grows as the shop grows	24 months

Each method pretends it is a few months ago, forecasts, and is checked against what really happened (the last 6 months). There is a **standard choice** for each history length, taken from my first evaluation. Another method replaces it only if it is **at least 30% more accurate**. This safety rule stops the system from changing models because of luck.

SIMPLE MEANING

Like testing five weather forecasters on last week's weather and hiring the best one, but only firing the usual one if the new one is clearly better.

Part 3 - Weekday calendar adjustment

From daily sales, the system learns how strong each weekday is for this shop. On the development branch it found **Saturday +33%** and **Tuesday -20%** compared with an average day. Each month gets a 'selling capacity' score from its number of days and how many strong weekdays it has. The

adjustment is kept only if it improves accuracy by at least 3%.

> **SIMPLE MEANING**
> February has 28 days and some months have five Saturdays. If Saturday is your best day, a month with five Saturdays should sell more. Normal models ignore this.

Part 4 - Honest ranges

The range (for example 'sales between 2.4M and 3.1M') is calculated from how wrong the chosen model was on the shop's last 8 months, separately for 1, 2, 3... months ahead. With few errors available, the range is made a little wider (Student-t correction).

> **SIMPLE MEANING**
> The old range was too narrow for next month (the real value was outside it too often) and far too wide for next year. Now the range is based on real mistakes, so it is more trustworthy and more useful.

Part 5 - Profit risk and break-even

- **Profit range** = sales forecast minus cost forecast, with a range. The system also measures how sales and cost errors move together (when sales are higher, ingredient costs are higher too).
- **Chance of a loss** = the probability that profit is below zero, for every month and for the 1, 3, 6 and 12-month totals.
- **Break-even**: costs are split into fixed costs (rent, salaries) plus a percentage of every sale (ingredients), learned from the last 12 months. Break-even sales = fixed costs / (1 - variable percentage).

> **SIMPLE MEANING**
> On the development branch: fixed costs about Rs. 424,878 a month, 32% of each sale goes on variable costs, so the shop needs about Rs. 623,519 of sales a month to cover all costs.

Part 6 - Owner-friendly output

The app turns the numbers into sentences, for example:

- 'Sales over the last 3 months are up 5% on the same months last year.'
- 'December is usually your busiest month (+27%) and February the quietest (-17%).'
- 'Saturday is your strongest day (+33%).'
- 'A loss in any of the next 3 months is unlikely.'
- 'Busiest month ahead: Dec 2026. Plan stock and staff early.'

Month pacing: 'By yesterday you would normally have Rs. X (59% of the month, counting your busy weekdays). You have Rs. Y.' It then shows **Ahead**, **On track** or **Behind**.

6. How I tested it

Data

- **S1** - 30 simulated restaurant branches, 36 months each (built from a realistic order-level simulation: growth, yearly season, weekday pattern, random noise).
- **S2** - the same branches with extra random shocks of +/-12% (harder test).
- **S3** - the development branch from the real app database.

Method: rolling-origin evaluation

For every month t, the model only sees the months before t, forecasts 1 to 12 months ahead, and the forecast is compared with what really happened. This is repeated for every possible month, so every result is a true out-of-sample test.

> **SIMPLE MEANING**
> It is like a mock exam: the model never sees the answers before it makes its prediction.

Fair tuning

The few settings (the 30% rule, how ranges extend) were tuned on **different** simulated branches (seeds 5000+). The final results below come from the original thesis branches (seeds 1000+), which were never used for tuning.

Measures (simple meaning)

Measure	Simple meaning	Good when
MAPE	Average % error. 7% = off by 7% on average	Lower
MASE	Error compared with a very simple guess. Below 1 = better than simple	Lower
Coverage	How often the real value fell inside the range (target 95%)	Near 95%
Range width	How wide the range is, as % of the real value	Narrower (if coverage OK)

7. Results

Sales forecast error (MAPE), old version vs new version

Dataset	Next month	3 months ahead	12 months ahead
S1 simulated	9.27% -> **7.22%**	8.89% -> **6.48%**	11.75% -> **8.59%**
S2 with shocks	12.17% -> **10.97%**	11.56% -> **10.26%**	14.39% -> **11.14%**
S3 dev branch	10.43% -> **8.06%**	9.88% -> **6.50%**	11.40% -> **8.66%**

Cost forecast error (MAPE), S1: next month 6.83% -> 5.96%, 3 months 6.51% -> 5.41%, 12 months 8.78% -> 7.29%.

Ranges (S1 sales)

	Next month	6 months ahead	12 months ahead
Old: coverage / width	81% / 35%	97% / 92%	99% / 147%
New: coverage / width	**92%** / 42%	97% / **71%**	98% / **92%**

SAY THIS TO THE PANEL

On every dataset and every horizon, the new version reduced the sales forecast error, by between 10 and 34 percent. For example, the 3-month error on simulated branches went from 8.9% to 6.5%. The next-month range now contains the real value 92% of the time instead of 81%, and the 1-year range is 92% of the actual value wide instead of 147%, so it is narrower and more useful.

Other evidence

- The weekday profile learned from data matched the simulation (Saturday +33% learned vs +32% built in). The break-even split matched too (32% variable cost learned vs 32% built in).
- On the development branch, the chosen model's back-test error was 3.42% for sales.
- Response time: about 1 second for a full forecast.
- Testing: 27 Python tests and 22 Node tests (including live tests with the real Python service), all passing.

8. What changed from my first version

First version	Improved version
Model chosen only by number of months	Methods tested on the shop's own months, safe 30% rule
Half-recorded months broke the forecast	Automatic data-quality check and repair
No calendar effect	Weekday calendar adjustment learned per shop
Range = theory (in-sample error x square root of months)	Range from real past forecast errors
Profit = one number	Profit range + chance of a loss
No break-even	Break-even sales level and comparison with next month
Numbers only	Plain-language tips and month pacing

SAY THIS TO THE PANEL

My own evaluation of the first version showed three weaknesses: the fixed model rule, broken forecasts after incomplete months, and unreliable ranges. The improved version fixes each one, and I measured the improvement with the same test design.

9. Live demo script

1. Open **Analytics** and scroll to **Sales & cost forecast**.
2. Tap the horizon chips (Next month, 3, 6, 12 months): 'Predictions update for each horizon.'
3. Sales and Costs cards: 'Each prediction has a range, not just one number.'
4. Profit card: 'Profit range and the chance of a loss.'
5. Break-even card: 'The shop needs this much sales per month to cover costs; next month is above it.'
6. 'What the forecast sees': read 2 tips, show the weekday bar chart.
7. This-month card: 'Ahead / On track / Behind, based on the shop's own weekday pattern.'
8. Month by month list, then the Model accuracy card: 'The app tested several methods and shows which one it chose and why.'

BE CAREFUL

Practice the demo before the viva. Make sure the Python service is running and the app is connected to the right branch, or the forecast card will show an error.

10. Limitations (say these yourself)

Panels respect students who know the limits of their work. Say these before they ask:

- **The evaluation uses simulated data plus one development branch.** SmartCost runs at three real businesses, but their data was not used yet because the owners' consent is needed.
- **The next-month range** contains the real value about 90% of the time; the target is 95%.
- **Most of the accuracy gain** comes from choosing the right method for each history length. Testing methods per shop adds a smaller improvement on top, and the weekday adjustment helps mostly for longer horizons.
- **The forecast is monthly**, so a brand-new shop needs 3 complete months first.
- **Break-even** assumes costs are fixed plus a percentage of sales; it is hidden when that does not fit the data.
- Holidays and Poya days are not included yet.

11. Future work

- Add Sri Lankan Poya days and public holidays from the official government calendar.
- Use daily data so new shops get a forecast in their first month.
- Save every forecast and compare it with what really happened, to measure real-world accuracy at the live shops (with consent).
- Learn from many shops together, so a new shop can borrow patterns from similar shops.
- Push notifications for loss risk or 'behind pace' warnings.

BE CAREFUL

Do NOT say the system uses Prophet, ARIMA or deep learning. It uses exponential smoothing (Holt / Holt-Winters), moving average and seasonal naive.
Do NOT quote 8.7% MAPE, 3 bakery managers, or a 14% waste reduction from the group paper. Those are not in the code or the evaluation.
Do NOT say it was tested on real customer data. Say: simulated branches plus the development branch.
Before the viva: the improved version must be committed, and the thesis (Chapters 3-6, Table 5.3) updated, so the report and the talk show the same numbers.

12. Likely panel questions and simple answers

Q: Why not ARIMA, Prophet or deep learning (LSTM)?

A: Small shops have only 3 to 36 monthly data points. Complex models need much more data and overfit. Exponential smoothing is the standard, well-tested choice for short monthly series, and I compared it against simple baselines to prove it adds value.

Q: What is new in your work? Holt-Winters is old.

A: The model itself is not the novelty. The novelty is the whole adaptive pipeline for micro-SME data: data-quality repair, per-shop model selection with a safety rule, weekday calendar adjustment, ranges from real errors, and loss probability with break-even, all inside a POS and explained in plain language.

Q: How do you know the model is not overfitted?

A: Every result is out-of-sample: the model never sees the months it predicts. And the settings were tuned on different simulated branches from the ones used for the final results.

Q: Why simulated data?

A: Real shops do not have three years of clean data yet, and using live shop data needs owner consent. The simulation is order-level and includes growth, seasons, weekday patterns and noise. I also tested on the development branch and a harder scenario with random shocks.

Q: What is MAPE? Is 7% good?

A: MAPE is the average percentage error. 7% means the forecast is off by about 7% on average. For comparison, simply repeating last month's value gives about 15% error on the same branches, and my first version gave about 9%.

Q: How do you detect bad data?

A: Each month is compared with the shop's own median month. Fewer than 40% of the normal orders, fewer than half the normal selling days, or no expenses means the month is flagged.

Q: How is the chance of a loss calculated?

A: Profit = sales - costs. The spread of profit comes from the sales and cost errors and how they move together. With a normal distribution, I calculate the probability that profit is below zero.

Q: How is break-even calculated?

A: A regression on the last 12 months splits costs into a fixed part and a percentage of sales. Break-even sales = fixed costs / (1 - that percentage).

Q: What happens if the Python service is down?

A: The API returns a clear error and the app shows 'Forecast unavailable' with a Try again button. Nothing crashes. This is tested.

Q: How fast is it?

A: About 1 second for a full forecast of sales, costs and profit.

Q: Why is the next-month range only 90% and not 95%?

A: The range is learned from the shop's last 8 months of errors, which is a small sample. It improved from 81% to about 90%. Reaching 95% needs more history or more conservative ranges; this is future work.

Q: Why do you need 24 months for Holt-Winters?

A: To learn a yearly pattern, the model must see each month at least twice. Before that, the system uses methods that need less data, so the owner still gets a forecast from 3 months.

13. One-page cheat sheet

SAY THIS TO THE PANEL

An adaptive forecasting engine inside a POS that adjusts to how much data a small shop has and how good it is, and turns forecasts into decisions: profit range, chance of loss, break-even and monthly pace.

Number	Meaning
9.27% -> 7.22%	Sales error next month, simulated branches (old -> new)
8.89% -> 6.48%	Sales error 3 months ahead, simulated branches
11.75% -> 8.59%	Sales error 12 months ahead, simulated branches
9.88% -> 6.50%	Sales error 3 months ahead, development branch
81% -> 92%	Next-month range contains the real value
147% -> 92%	Width of the 1-year range (narrower = more useful)
8,000%+	Error of the old model after one half-recorded month (now detected)
30%	A new method must beat the standard one by 30% to replace it
5 methods	Moving average, seasonal naive, Holt, Holt-Winters add / mult
30 + 30 + 1	Simulated branches, shocked branches, development branch
Rs. 623,519	Break-even sales per month on the development branch
Saturday +33%	Strongest weekday learned on the development branch
27 + 22	Python tests + Node tests, all passing
~1 second	Time for a full forecast

The 6 parts in one line each

1. Data-quality check - finds and repairs incompletely recorded months.
2. Adaptive model choice - tests 5 methods, keeps the best with a safety rule.
3. Weekday calendar - learns busy days; more strong days = higher forecast.
4. Honest ranges - built from the model's real past mistakes.
5. Profit risk and break-even - profit range, chance of loss, sales needed to cover costs.
6. Owner-friendly output - plain-English tips and ahead / on track / behind.